

Single Port Ram Report

A. Encountered Bugs

1. Zero-Time Execution in Driver (Timing Bug)

- **Symptom:** The driver popped all stimulus from the sequence queue instantly, overwriting the DUT pins in zero simulation time. Only the very last transaction in the queue was actually captured by the DUT.
- **Root Cause:** The drive() function utilized a while (!input_queue.empty()) loop. Because C++ executes infinitely faster than the simulated Verilator clock, the loop drained the entire queue before the clock could advance.

```
void Driver::drive() {  
    while (!input_queue.empty()) {  
        RamItem item = input_queue.front();  
        input_queue.pop();  
  
        // Drive the DUT with the input item  
        dut->addr = item.addr;  
        dut->din = item.din;  
        dut->cs = item.cs;  
        dut->we = item.we;  
  
        gm->get_golden_input(item);  
    }  
}
```

- **Solution:** Replaced the while loop with an **if statement** to ensure cycle-accurate execution (processing exactly one transaction per clock cycle) and separated the stimulus generation phase into a dedicated Sequence class.

2. Persistent State Overwrite (Missing Idle State)

- **Symptom:** After the stimulus queue was empty, the simulation continued to drive the last known transaction (e.g., repeatedly writing to the same address / reading from the same address).

- **Root Cause:** Hardware pins hold their previous state. The drive() function lacked logic to handle an empty queue, so whatever values were left on dut->cs and dut->we remained asserted for the rest of the simulation.
- **Solution:** Added an else block to the drive() method to actively drive idle/default values (dut->cs = 0; dut->we = 0;) when no new transactions are present.

```
void Driver::drive() {
    if (!input_queue.empty()) {
        RamItem item = input_queue.front();
        input_queue.pop();

        // Drive the DUT with the input item
        dut->addr = item.addr;
        dut->din = item.din;
        dut->cs = item.cs;
        dut->we = item.we;

        gm->get_golden_input(item);
    }
    else {
        dut->cs = 0;
        dut->we = 0;
    }
}
```

3. Segmentation Fault at Simulation Start (Memory Violation)

- **Symptom:** The simulation crashed with a Segmentation fault (core dumped) immediately after the setup phase.

```
◎ thanhtai@Lenovo-L0Q-15IRX9:~/Documents/Verilog_Projects/Synchronous_RAM_CPP$ make run
./obj_dir/Vtop
-----
SIMULATION STARTING WITH SEED: 1780445401
-----
make: *** [Makefile:51: run] Segmentation fault (core dumped)
✧ thanhtai@lenovo-l0q-15irx9:~/Documents/Verilog_Projects/Synchronous_RAM_CPP$ █
```

- **Root Cause:** An uninitialized pointer. The Scoreboard* scb pointer was passed into the Monitor constructor but was never assigned to the internal class member. When the Monitor attempted to call scb->write(), it accessed a garbage memory address.
- **Solution:** Updated the Monitor constructor's initializer list to properly bind the pointer (scb(_scb)).

6. False Read-After-Write Mismatches (0-Delay Race Condition)

- **Symptom:** The Scoreboard reported false data mismatches specifically when a Read immediately followed a Write to the same address.

```
Write At Address 0x5 -> Data : 0xb
DATA MATCH AT 0x2 -> EXP : 0xc || ACT : 0xc
Write At Address 0x2 -> Data : 0x8
----> DATA MISMATCH AT 0x2 -> EXP : 0x8 || ACT : 0xc
Write At Address 0x1 -> Data : 0xb
DATA MATCH AT 0xd -> EXP : 0xc || ACT : 0xc
Write At Address 0xf -> Data : 0x0
DATA MATCH AT 0x1 -> EXP : 0xb || ACT : 0xb
```

- **Root Cause:** An execution order flaw in the main simulation loop. The Driver applied the new stimulus and instantly updated the Golden Model *before* the Scoreboard could verify the previous cycle's Read transaction. The Scoreboard ended up comparing the old RAM output against the newly updated Golden Model state.

```
//=====
// Pre-active phase (Before the rising clock)
//=====
mon->sample();

drv->drive();

scb->compare();

dut->eval();

contextp->timeInc(5);
tfp->dump(contextp->time());
```

- **Solution:** Swapped the execution order in sim_main.cpp. The sequence was corrected to:
 1. Monitor samples previous cycle.
 2. Scoreboard compares previous cycle.
 3. Driver applies new cycle stimulus and updates Golden Model.

B.Functional Coverage

CUMULATIVE COVERAGE SUMMARY		
Total Runs Executed : 100		
Passing Runs : 100/100		

[100%]	CS=1 Hits	: 4998/1
[100%]	CS=0 Hits	: 2/1
[100%]	Write Hits	: 2628/1
[100%]	Read Hits	: 2622/1
[100%]	Unique Addresses Tested	: 16/16
[100%]	Unique Data Values Tested	: 16/16
[100%]	Valid Cross Combinations (CS=1)	: 512/512
=====		
FINAL FUNCTIONAL COVERAGE: 100.0%		
=====		